KAVACH

Field Worker Execution Agent

Detailed Technical Flowcharts & Beginner-Friendly Explanation

A deep implementation guide for task receipt, navigation, before/after photos, GPS capture, checklist progress, status transitions, completion submission and failure handling.

Purpose	Let a worker execute an assigned task and submit structured progress, location and photographic evidence.
Prototype stack	React worker UI when needed, Python, FastAPI, Pydantic, temporary in-memory repositories and simulated/local image storage.
Core principle	A completion submission is a worker claim with evidence; it is not automatically independently verified.
Scope	Field Worker stage only, based on the uploaded diagram's stated inputs and outputs.

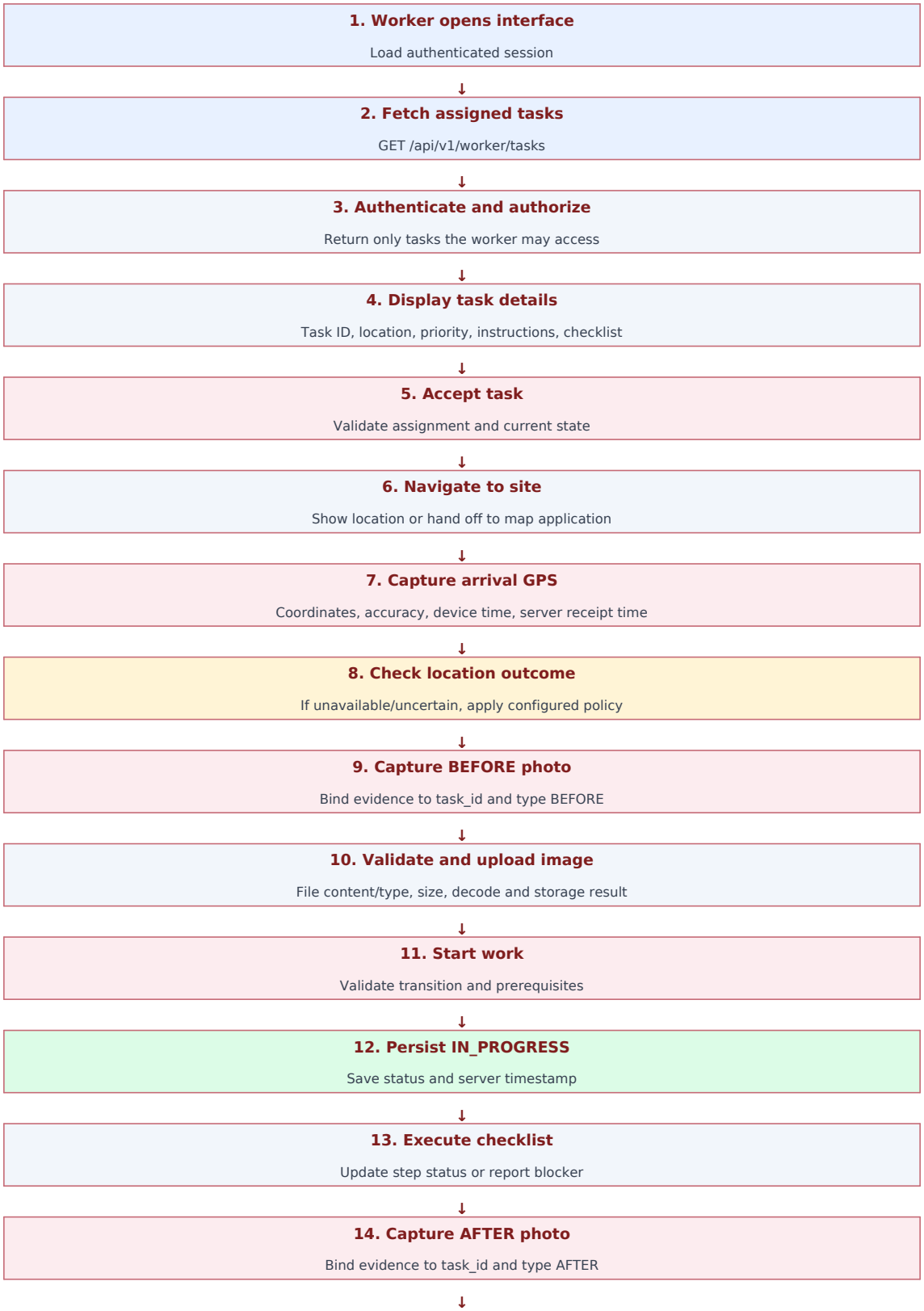
Endpoint names, state labels, sample data and implementation choices are proposed designs, not confirmed existing KAVACH APIs or policies. Confirm them in the shared API contract before parallel implementation.

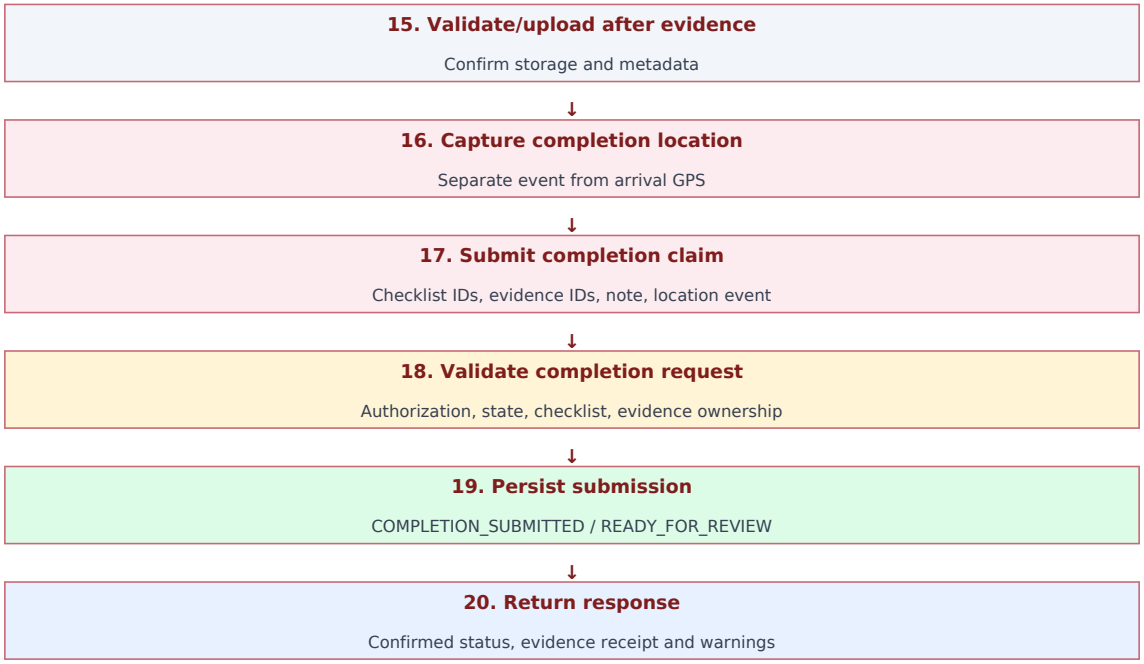
1. Understand the stage

The earlier stages create a plan and assign a task. This stage is where the person performs the real-world work. KAVACH displays the assignment, records progress, captures evidence and preserves a reliable history. Example: the worker opens the task, reaches the site, records arrival GPS if available, captures a before photo, starts work, follows the checklist, captures an after photo and submits completion.

The worker interface sends requests; the backend is authoritative for permissions, evidence records and task state.

2. Master technical flowchart





Main path. Never invent coordinates or claim that an upload succeeded before server confirmation.

3. Decision and error flowchart

Worker authorized for this task?

No → reject without exposing protected task details.

↓

Task state permits acceptance/start?

No → return conflict and current state.

↓

Location available and usable?

No → retry or record unavailable; continue only if policy allows.

↓

Image supported and safely decodable?

No → reject and allow recapture.

↓

Evidence stored successfully?

No → mark pending/failed; do not claim it is stored.

↓

Required before evidence present?

No → block start if policy requires it.

↓

Checklist and after evidence complete?

No → return missing items or permitted exception.

↓

Completion matches current state?

No → reject conflict or return prior idempotent result.

↓

All validation passes?

Yes → persist completion submission; otherwise return field-level errors.

4. Detailed explanation of each stage

A. Retrieve assigned tasks

Use an authenticated endpoint such as GET /api/v1/worker/tasks. Derive worker identity from the authenticated session/token; do not trust arbitrary client-supplied worker IDs. Return only authorized tasks.

B. Show task instructions

Display task ID, location, priority, checklist and resource requirements. The UI displays server state; it does not create authoritative state by changing a label.

C. Accept task

Send an acceptance request. Verify assignment, worker authorization and current state. If another update already changed the task, return a conflict and refresh.

D. Navigate

Display the location or hand off to a maps app. Navigation is not proof of arrival. Week 1 can use synthetic coordinates or text locations.

E. Capture GPS

Request permission and record latitude, longitude, accuracy, event type (ARRIVAL/COMPLETION), optional device timestamp and server receipt time. A coordinate is evidence, not absolute proof of presence.

F. Capture BEFORE photo

Open camera/file picker, associate the image with task ID and evidence type BEFORE, validate file content/size/type, upload it and record the server storage result.

G. Start work

Backend checks the allowed transition and prerequisites, then persists IN_PROGRESS and a server timestamp. A local UI change is not enough.

H. Track checklist

Record each step's status. Allow a blocker/pause only through explicit rules. Avoid continuous GPS tracking unless there is a justified requirement.

I. Capture AFTER photo

Upload as AFTER and verify that the evidence belongs to the task. A file hash helps identify exact duplicate bytes; it does not prove that the image depicts the actual site or successful work.

J. Completion location

Record a separate event rather than overwriting arrival coordinates. Preserve accuracy and unavailable/uncertain states.

K. Submit completion

Submit checklist IDs, before/after evidence IDs, optional note and location event reference. Use stable IDs/idempotency so retries do not duplicate records.

L. Validate and persist

Check authorization, current state, required checklist items, evidence ownership and location requirements. Save the submission in a controlled operation.

M. Return confirmed status

A successful response confirms that the submission was recorded, not that the work was independently verified.

5. Proposed task state machine

```
stateDiagram-v2
    [*] --> ASSIGNED
    ASSIGNED --> ACCEPTED: worker accepts
    ASSIGNED --> REJECTED: worker rejects
    ACCEPTED --> IN_PROGRESS: start accepted
    IN_PROGRESS --> PAUSED: blocker reported
    PAUSED --> IN_PROGRESS: work resumes
    IN_PROGRESS --> COMPLETION_SUBMITTED: evidence/checklist submitted
    COMPLETION_SUBMITTED --> IN_PROGRESS: returned for correction
    REJECTED --> [*]
    COMPLETION_SUBMITTED --> [*]: field-work stage ends
```

This is a proposed lifecycle. Define exact statuses in the shared API contract and enforce transitions on the backend. Reject transitions that do not match the current state, and protect concurrent updates.

Example transition validator

```
ALLOWED_TRANSITIONS = {
    "ASSIGNED": {"ACCEPTED", "REJECTED"},
    "ACCEPTED": {"IN_PROGRESS"},
    "IN_PROGRESS": {"PAUSED", "COMPLETION_SUBMITTED"},
    "PAUSED": {"IN_PROGRESS"},
    "COMPLETION_SUBMITTED": set(),
    "REJECTED": set(),
}

def can_transition(current, new):
    return new in ALLOWED_TRANSITIONS.get(current, set())
```

6. Suggested API endpoints

Method	Endpoint	Purpose
GET	/api/v1/worker/tasks	List authorized tasks
GET	/api/v1/worker/tasks/{task_id}	Fetch task and checklist
POST	/api/v1/worker/tasks/{task_id}/accept	Accept assignment
POST	/api/v1/worker/tasks/{task_id}/arrival	Record arrival location event
POST	/api/v1/worker/tasks/{task_id}/evidence	Upload BEFORE/AFTER evidence
POST	/api/v1/worker/tasks/{task_id}/start	Start work
PATCH	/api/v1/worker/tasks/{task_id}/checklist/{step_id}	Update checklist step
POST	/api/v1/worker/tasks/{task_id}/pause	Report blocker
POST	/api/v1/worker/tasks/{task_id}/completion	Submit completion claim

7. Example contracts and evidence models

Illustrative completion request

```
{
  "task_id": "TASK-104",
  "completion_note": "Overflow collected and path cleaned",
  "checklist_completed": ["STEP-01", "STEP-02", "STEP-03"],
  "before_evidence_ids": ["EVD-501"],
  "after_evidence_ids": ["EVD-502"],
  "completion_location_event_id": "LOC-902"
}
```

Illustrative response

```
{
  "task_id": "TASK-104",
  "status": "COMPLETION_SUBMITTED",
  "submission_id": "SUB-104",
  "evidence_status": "ACCEPTED",
  "location_status": "CAPTURED",
  "message": "Completion submitted for review"
}
```

Only return CAPTURED when location capture actually succeeded. Otherwise return a truthful status such as UNAVAILABLE or NOT_REQUIRED.

EvidenceRecord fields

```
evidence_id
task_id
submitted_by_worker_id
evidence_type: BEFORE | AFTER
storage_key (private)
validated_mime_type
file_size_bytes
sha256 (integrity aid)
uploaded_at_server
upload_status: PENDING | STORED | FAILED

LocationEvent fields
location_event_id, task_id, worker_id
event_type: ARRIVAL | COMPLETION
latitude, longitude (nullable)
accuracy_meters (nullable)
captured_at_device (optional/untrusted)
received_at_server
capture_status: CAPTURED | UNAVAILABLE | INACCURATE
```

Store image bytes in private file/object storage and metadata in the database. Restrict access to precise location information and avoid logging raw coordinates unnecessarily.

8. Failure handling

Situation	Expected behavior
Unsupported/corrupt image	Reject safely and allow recapture.
Upload fails	Mark pending/failed; do not claim server storage succeeded.
Location permission denied	Record unavailable and apply configured policy.
Network fails after upload	Refresh and retry idempotently; avoid duplicate evidence.
Task already submitted	Return current/prior result safely; avoid duplicate state transition.
Evidence belongs to another task	Reject mismatched IDs.
Checklist incomplete	Return missing step IDs or permitted exception path.
Worker unauthorized	Reject without exposing protected task data.

9. Security and reliability

- Authenticate worker and authorize every task, evidence and location operation.
- Validate actual image content, size and decoding; do not trust filename extension alone.
- Store images privately; use controlled access when images are retrieved.
- Capture only task-relevant location events unless continuous tracking has a justified need.
- Keep server timestamps and treat device timestamps as untrusted evidence.
- Use stable evidence IDs and idempotency to make retries safe.
- Keep an audit trail of state changes and evidence submission.
- Distinguish captured locally, upload pending, server-stored and submitted for review.

10. Suggested code structure

```
app/
  main.py
  api/
    worker_task_routes.py
    evidence_routes.py
    location_routes.py
  schemas/
    worker_task.py
    checklist.py
    evidence.py
    location_event.py
    completion.py
  services/
    field_work_service.py
    task_state_service.py
    evidence_service.py
    location_capture_service.py
    completion_submission_service.py
  repositories/
    task_repository.py
    evidence_repository.py
    location_repository.py
  storage/
    image_storage.py
  validators/
    task_transition_validator.py
    image_validator.py
    completion_validator.py
tests/
  test_worker_task_routes.py
  test_task_state_transitions.py
  test_evidence_upload.py
  test_location_capture.py
  test_completion_submission.py
```

11. Testing and Week 1 build order

Test	Expected result
Authorized worker fetches tasks	Only permitted assignments are returned.
Unauthorized worker accesses task	Request is rejected.
Start without required before evidence	Configured prerequisite is enforced.
Invalid image content	Upload rejected safely.
GPS unavailable	Explicit unavailable/exception status.
Missing checklist item at completion	Submission rejected or exception path used.
Completion references wrong evidence	Submission rejected.
Repeated completion request	No duplicate completion record.
Conflicting concurrent status updates	Only a valid transition is committed.

First — agree on contracts

Define task, evidence, location event, completion schemas and exact status transitions.

Second — implement state machine

Test permitted and rejected transitions using in-memory task data.

Third — evidence upload

Validate small test files and record storage results.

Fourth — GPS events

Accept coordinates/accuracy and explicitly handle unavailable values.

Fifth — completion submission

Validate checklist and evidence references and persist a completion claim.

Sixth — test failure paths

Cover authorization, invalid images, retries, stale state and duplicate requests.

Definition of done

API tests demonstrate task receipt, before/after evidence, location events, valid status transitions and truthful completion submission.

Final summary

The worker fetches and accepts a task, navigates to the site, records arrival GPS where available, uploads before evidence, starts work, updates checklist progress, uploads after evidence, records completion location and submits a completion claim. FastAPI authenticates and validates requests; Python enforces task-state rules; evidence and location records are saved with metadata. The system reports only what the server has actually confirmed.